

# Karma でのレンダリング – ノイズはサンプル数で本当に減るのか

2026-09-13 | Houdini 21.0.700 | /obj/pyro\_test | ノード 6 / 接続 5

## 何を作ったか

実験016で煙を描いたとき、ハードウェアレンダラー（OpenGL ROP）では濃淡が分らないと書いた。そこで本式のレンダラーである Karma を使う。ただ「きれいになった」で終わらせず、測れるものを測る。

Karma は1ピクセルの色を、乱数で飛ばした何本もの光線の平均で決める。平均のばらつきは本数の平方根に反比例するはずなので、サンプル数を4倍にすればノイズは半分になるという式が立つ。これを実測で確かめた。

結果、式が当たるのは狭い範囲だけだった。そして016で書いた「ハードウェアレンダラーはボリウムを描けない」も、言い過ぎだったと分かった。

## ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード       | タイプ                       | デフォルトから変更したパラメータ                                                       |
|-----------|---------------------------|------------------------------------------------------------------------|
| emitter   | sphere                    | radx = 0.5<br>rady = 0.5<br>radz = 0.5<br>ty = 0.6<br>type = 1         |
| source    | pyrosource                | attribute1 = 0<br>attributes = 1<br>mode = 2                           |
| rasterize | volumerasterizeattributes | attributes = density<br>folder1 = 1<br>folder3 = 1<br>voxelsize = 0.08 |

| ノード        | タイプ        | デフォルトから変更したパラメータ                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| solve_d0_0 | pyrosolver | _folder32_1 = 1<br>color_diss_controlramp1interp = 3<br>color_diss_controlramp2interp = 3<br>color_diss_controlramp2pos = 1.0<br>color_diss_controlramp2value = 1.0<br>density_ramp2pos = 1.0<br>density_ramp2value = 1.0<br>dissipation = 0.0<br>dissipation_controlramp2pos = 1.0<br>dissipation_controlramp2value = 1.0<br>dissipation_remapcontrol = 1<br>disturbance_controlramp2pos = 1.0<br>…ほか 102 件                   |
| solve_d0_1 | pyrosolver | _folder32_1 = 1<br>color_diss_controlramp1interp = 3<br>color_diss_controlramp2interp = 3<br>color_diss_controlramp2pos = 1.0<br>color_diss_controlramp2value = 1.0<br>density_ramp2pos = 1.0<br>density_ramp2value = 1.0<br>dissipation_controlramp2pos = 1.0<br>dissipation_controlramp2value = 1.0<br>dissipation_remapcontrol = 1<br>disturbance_controlramp2pos = 1.0<br>disturbance_controlramp2value = 1.0<br>…ほか 100 件 |
| solve_d0_3 | pyrosolver | _folder32_1 = 1<br>color_diss_controlramp1interp = 3<br>color_diss_controlramp2interp = 3<br>color_diss_controlramp2pos = 1.0<br>color_diss_controlramp2value = 1.0<br>density_ramp2pos = 1.0<br>density_ramp2value = 1.0<br>dissipation = 0.3<br>dissipation_controlramp2pos = 1.0<br>dissipation_controlramp2value = 1.0<br>dissipation_remapcontrol = 1<br>disturbance_controlramp2pos = 1.0<br>…ほか 101 件                   |

## サンプル数を変えたときのノイズ

ノイズの量は、基準画との差ではなく、3枚を連立させて1枚ずつ求めた。基準画にもノイズが残っている  
ので、基準画に頼ると結果がその出来に左右されるため。640×360、煙とみなした画素は 21,525 個。

| サンプル数 | ノイズ    | 前段との比  | 式の予測   | ずれ     |
|-------|--------|--------|--------|--------|
| 2     | 18.316 | —      | —      | —      |
| 4     | 11.321 | 0.6181 | 0.7071 | −12.6% |
| 8     | 6.516  | 0.5756 | 0.7071 | −18.6% |
| 16    | 4.559  | 0.6997 | 0.7071 | −1.0%  |
| 32    | 3.413  | 0.7486 | 0.7071 | +5.9%  |
| 64    | 2.640  | 0.7734 | 0.7071 | +9.4%  |
| 128   | 2.345  | 0.8883 | 0.7071 | +25.6% |

| サンプル数 | ノイズ   | 前段との比  | 式の予測   | ずれ     |
|-------|-------|--------|--------|--------|
| 256   | 2.088 | 0.8904 | 0.7071 | +25.9% |

サンプル数を4倍にしたとき（式なら 0.5000）

8→32 の区間だけが式どおり。その下では式より速く、その上では遅く減る。

| 区間       | 実測の比   | 式との差     |
|----------|--------|----------|
| 2 → 8    | 0.3558 | 式より速く減る  |
| 4 → 16   | 0.4028 | 式より速く減る  |
| 8 → 32   | 0.5238 | ほぼ式どおり   |
| 16 → 64  | 0.5789 | やや遅い     |
| 32 → 128 | 0.6870 | 明らかに遅い   |
| 64 → 256 | 0.7909 | ほとんど減らない |

指定したサンプル数と、実際に使われたサンプル数

2～256 の範囲で時間を直線に当てはめると「固定費 2.26秒 + 1サンプルあたり 7.44ミリ秒」になる。この直線から逆算した実効サンプル数を右端に置いた。指定を64倍にしても、実際には6倍にもなっていない。

| 指定   | 実測の時間  | 直線の予測   | 実測/予測 | 実効サンプル |
|------|--------|---------|-------|--------|
| 64   | 3.00 秒 | 2.74 秒  | 1.10  | 99     |
| 128  | 3.09 秒 | 3.21 秒  | 0.96  | 112    |
| 256  | 3.43 秒 | 4.16 秒  | 0.82  | 157    |
| 512  | 4.23 秒 | 6.07 秒  | 0.70  | 264    |
| 1024 | 5.52 秒 | 9.88 秒  | 0.56  | 439    |
| 2048 | 5.94 秒 | 17.50 秒 | 0.34  | 495    |
| 4096 | 6.48 秒 | 32.73 秒 | 0.20  | 567    |

頭打ちの犯人 — varianceaa\_thresh

samplesperpixel を 4096 に固定したまま、打ち切りのしきい値だけを下げた。しきい値を50分の1にすると、実際の仕事量は21.9倍になった。頭打ちはハードウェアの限界ではなく、この設定によるもの。

| varianceaa_thresh | 時間      | 実効サンプル | 既定との比 |
|-------------------|---------|--------|-------|
| 0.01（既定）          | 5.39 秒  | 420    | 1.00  |
| 0.005             | 6.29 秒  | 542    | 1.29  |
| 0.001             | 8.01 秒  | 773    | 1.84  |
| 0.0002            | 70.64 秒 | 9,191  | 21.87 |

同じ煙を2つのレンダラーで — 同じカメラ・同じ露出

「煙の画素」は真っ白でない画素の数。「最も暗い」は、その画素がどこまで濃く描かれたか。

| レンダラー            | 時間     | 煙の画素   | 階調    | 平均の明るさ | ばらつき  | 最も暗い |
|------------------|--------|--------|-------|--------|-------|------|
| OpenGL ROP       | 0.71 秒 | 29,242 | 235 段 | 101.0  | 86.16 | 15   |
| Karma<br>256サンプル | 3.64 秒 | 21,545 | 200 段 | 109.0  | 52.83 | 50   |

## 煙の中心を横切る線の明るさ

左から右へ等間隔に見たときの値。255 が背景。数字が小さいほど濃い。

| レンダラー      | 縁の手前 | 縁   | 中へ  | 中心  | 中心  | 中へ  | 縁   | 縁の外 |
|------------|------|-----|-----|-----|-----|-----|-----|-----|
| OpenGL ROP | 254  | 224 | 121 | 58  | 42  | 50  | 112 | 219 |
| Karma      | 255  | 255 | 184 | 109 | 104 | 104 | 189 | 255 |

## わかったこと

- ・式が当たったのは 8~32 の区間だけだった。サンプル数を4倍にしたときの比は、8→32 で 0.5238（式は 0.5000）とよく一致する。ところが 64→256 では 0.7909 で、ほとんど減らない。

- ・減らなくなる理由を突き止めた。samplesperpixel  
は「ここまで使ってよい」という上限であって、「必ずこれだけ使う」という指定ではない。指定を 64 から 4096 へ64倍にしても、時間から逆算した実効サンプル数は 99 から 567 へ5.7倍にしかならなかった。

- ・犯人は varianceaa\_thresh（既定 0.01）だった。これは「このくらい収束したら、そのピクセルはもう打ち切る」というしきい値。4096 を指定したまましきい値を 0.0002 まで下げると、実効サンプル数は 420 から 9,191 へ21.9倍になり、時間も 5.39秒から70.64秒に伸びた。上限ではなく打ち切りの設定が効いていた。

- ・実用的な結論。この場面では 32 サンプルあたりで費用対効果が折り返す。それ以上は、数字を上げるだけでは絵は良くなる。本当に詰めたいときは varianceaa\_thresh も一緒に下げる。ただし時間は一気に13倍になる。

- ・時間の内訳も分かった。640×360 で固定費が 2.26 秒、1サンプルあたり 7.44 ミリ秒。2サンプルでは時間のほぼ全部が固定費で、256サンプルでも半分強が固定費だった。軽い絵を何枚も撮るときは、サンプル数より枚数のほうが効く。

- ・実験016に書いた「ハードウェアレンダラーはボリュームを描けない」は言い過ぎだった。同じカメラ・同じ露出で撮り直すと、OpenGL ROP も煙の輪郭と、厚みによる濃淡（中心ほど濃い）を描けている。中心を横切る線を見ると、縁の 254 から中心の 42 までなだらかに変化している。

- ・2つのレンダラーの本当の違いは、光が中を通るかどうかが違った。OpenGL は濃さの分だけ暗くするだけで、どちらから光が当たっているかが絵に出ない。Karma は光源側が明るく、反対側が暗い。Karma のほうが最も暗い画素が 50 と薄いのも、光が内部を通過して抜けてくるため。

- ・速さの差は大きい。OpenGL が 0.71 秒に対し、Karma は 256サンプルで 3.64 秒。5倍。確認のための描画は今後も OpenGL でよく、Karma は見せる絵に使う。

- ・測り方で3回つまずいた。(1) 出力画像には透明度が付いていて、透明部分の色は黒だった。白い背景に見えるのは表示側が白で合成しているため。これに気づかず「白から離れた画素＝煙」というマスクを作り、画面の88%を拾って、ほとんど何も測っていなかった。

- ・ (2) 煙の芯が真っ白 (255) に張り付いていた。張り付いた画素は元の値が分からないので、ノイズを測っても意味がない。ライトを下げ、中間調の画素だけを対象にした。

- ・ (3) OpenGL と Karma でカメラの寄り方が違っていた (margin 1.12 対 0.5) 。煙の写る画素数が違って当たり前で、最初の比較は無効だった。実験003以降ずっと守ってきた「全変種を同じカメラで撮る」を、自分で破っていた。

- ・ Apprentice でも Karma

は問題なく動く。出力には右下にウォーターマークが入るので、測る範囲からは外している。

## 次にやること

- ・ 構造のある煙を作って撮り直す。いまの煙はほぼ一様な球で、レンダラーの差を見せる題材として弱い

- ・ 018で決めた Karma の設定 (denoiser off、varianceaa\_thresh、露出) を hou\_tools に関数としてまとめる

- ・ 019 として FLIP に再挑戦する。付属ヘルプで flipsolver の入力の作法を読む